

Azure AI Solutions for Master Patient Index

Below is a concise, work-focused analysis of the arXiv paper your architect pointed to: “**From Local to Global: A GraphRAG Approach to Query-Focused Summarization**” (arXiv:2404.16130). I’ll highlight what it *is*, what it *claims*, how it *works*, and—most importantly—how (and whether) it applies to your **Master Patient Index (MPI)** / **patient linkage** problem.

1) What this paper is about (in one paragraph)

The paper introduces **GraphRAG**, a graph-based *retrieval-augmented generation* method intended to answer **global sensemaking questions** (e.g., “what are the main themes across the corpus?”), where classic “vector RAG” (chunk → embed → top-k retrieve) struggles because relevant evidence is **distributed across the collection** rather than localized in a few chunks. GraphRAG constructs an **entity knowledge graph** from source documents and then generates **community summaries**; at query time it composes **partial answers per community** and finally merges them into a global answer. The authors report **substantial improvements** over a baseline RAG for **comprehensiveness and diversity** on datasets ~1M tokens.

2) Core method & pipeline (what GraphRAG actually does)

Indexing time (two stages):

1. **Entity graph construction** — Use LLM prompts to extract entities/relations (“nodes/edges/covariates”) from text; build a **knowledge graph** capturing connections across the corpus.
2. **Community detection & summarization** — Partition the graph (e.g., **Leiden** algorithm) into communities of closely-related entities; **pre-generate summaries** per community for later reuse.

Query time: For a user question, **select relevant community summaries**, generate **partial responses** per community, and **summarize them** into a final global answer.

The authors emphasize GraphRAG targets **query-focused summarization** over large private text corpora (e.g., internal collections), not just local snippet retrieval. They position “vector RAG” as effective for localized facts, but insufficient for **global sensemaking** across an entire dataset; hence the graph + community summaries design.

3) Claimed results & caveats

- **Claims:** On “global sensemaking” tasks over ~1M tokens, GraphRAG improves **comprehensiveness** (covers more relevant aspects) and **diversity** (less repetitive, broader perspectives) compared to a conventional RAG baseline. The paper publishes a method, not a product; an open-source implementation is referenced.
 - **Caveats (independent critique):** A subsequent study argues many GraphRAG evaluations have **biases** (e.g., question choice and LLM-as-judge pitfalls), and when those are addressed the performance gains may be **more moderate** than initially reported. Treat published gains as **directional**, not absolute, and validate on *your* datasets.
-

4) Fit to your patient linkage / MPI problem

Important distinction: GraphRAG is a QFS/RAG technique for answering *questions over document corpora*. Your current need—linking patient records across EHRs to determine “same patient”—is a **record linkage** problem requiring **decision rules** and **similarity scoring** (deterministic + probabilistic), auditability, and **human-in-the-loop resolution** when confidence is ambiguous. That is **not** what GraphRAG solves directly.

Where GraphRAG **can help** in healthcare data contexts:

- **Global sensemaking & cohort exploration:** Once you have an MPI (a *Golden ID* mapping), GraphRAG can summarize **themes** across linked records or across provider documentation, e.g., “What common errors cause duplicates?” or “Which demographic clusters drive most merges/unmerges?”—global questions that benefit from community summaries over an entity graph.
- **Knowledge graph enrichment:** You *can* use its **graph-building prompts** and **community detection** ideas to construct a **patient entity graph** (patients, identifiers, addresses, encounters). But you’ll **still need a linkage scoring policy** (deterministic IDs, probabilistic similarities, thresholds, and survivorship rules) to decide **same/different**—GraphRAG doesn’t specify that.

This aligns with what we’ve discussed internally—using **field weighting** and **confidence thresholds** for deduplication and requiring human review under gray-zone scores—typical MPI practice rather than GraphRAG’s QFS emphasis.

5) Practical architecture guidance (grounded to your stack)

Given your plan to store **patient data in Azure Health Data Services (FHIR)** and **export to Fabric bronze/silver** for analytics, here’s a pragmatic split of responsibilities:

A. MPI / linkage subsystem (decisioning) — core to your use case

- **Input:** Fabric **silver** (relationalized FHIR Patient, Encounter, etc.).
- **Scoring:** Deterministic matches (Enterprise ID, MRN overlaps, DOB exact, phone/email exact), **probabilistic** similarities (names: phonetic + Jaro-Winkler; addresses: token & geospatial; optional **embedding cosine** for profile text).
- **Thresholds:** Auto-merge (e.g., ≥ 0.85), human-review (e.g., $0.65\text{--}0.85$), no-match (< 0.65).
- **Ledger:** Persist **Golden ID** ↔ **source IDs**, maintain **provenance and unmerge capability**.

This is classic EMPI logic and should live in your **graph DB** (Neo4j/CosmosDB Gremlin) or **relational store**, driven by **Spark/Functions**. (You outlined similar weighting rules in earlier talks.)

B. GraphRAG subsystem (sensemaking & QA over corpora)

- **Use cases:** “Explain duplicate patterns by site,” “summarize community-level trends,” “surface policies referenced across merges.”
 - **Implementation:** Build an **entity knowledge graph** (patients, identifiers, facilities, claims, notes) and **pre-compute community summaries**; at query time, assemble **partial community answers** → **global answer**.
 - **Tooling references:** official **Microsoft Research** publication and project site; plus community implementations (LangChain GraphRAG) if you need framework integration.
-

6) Key takeaways (so you can respond to the architect)

1. **GraphRAG ≠ patient matching.** It targets **global summarization over corpora**, not **pairwise linkage scoring**. For MPI you still need **deterministic+probabilistic** decisioning with thresholds and audit trails.

2. **Use GraphRAG where it shines: sensemaking questions** over your entire dataset (themes, patterns, cohorts). It's complementary to, not a replacement for, an EMPI.
 3. **Validate performance claims on your data.** Independent work notes **evaluation biases**; run **controlled A/B** tests against your corpus (e.g., Kenvue R&D, Pacifico Salud) before committing.
-

7) Suggested next steps (fastest path forward)

- **Keep** the MPI scoring spec you're drafting (deterministic IDs + probabilistic similarities + thresholds + human review). It maps to requirements we've been discussing in [Pacifico Salud InterOperability - Strategy and Deployment Discussion-20251121 140357-Transcripción de la reunión.mp4](#) (weighting + human validation).
 - **Add** a small GraphRAG PoC over your **silver** lakehouse to answer global questions (e.g., "Which identifier combinations most commonly lead to false merges?"). Use the official paper and MSR resource as build guidance.
 - **Evaluate claims** with your data: Create a benchmark of **global questions**, compare **GraphRAG vs vector RAG**, and assess **comprehensiveness/diversity**—being mindful of the evaluation caveats.
-

References (for your reply and internal deck)

- ArXiv page & PDF (v2, Feb 19, 2025): "[From Local to Global: A GraphRAG Approach to Query-Focused Summarization]"(<https://arxiv.org/abs/2404.16130>) and "[PDF]"(<https://arxiv.org/pdf/2404.16130v2>).
 - HTML version & method details (community detection, pipeline diagrams): "[HTML v2]"(<https://arxiv.org/html/2404.16130v2>) and "[HTML v1]"(<https://arxiv.org/html/2404.16130v1>).
 - Microsoft Research publication page: "[MSR: GraphRAG]"(<https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>).
 - Independent evaluation critique: "[How Significant Are the Real Performance Gains? An Unbiased Evaluation Framework for GraphRAG]"(<https://arxiv.org/html/2506.06331v1>).
-

Quick check: Do you want me to turn this into (A) a **one-pager MPI scoring spec** for review with Rajesh Singh (Applied AI) and [Ethan Jenkins](#), and (B) a **GraphRAG PoC plan** over your **FHIR silver** lakehouse with concrete tasks and owners?

Here's how scoring logic typically works for **patient linkage in a graph database**. The goal is to compute a confidence score for whether two nodes (patients) represent the same individual, based on their attributes and relationships.

✓ Core Principles

- Each patient node has attributes (name, DOB, gender, phone, email, address).
 - Relationships connect patients to identifiers, addresses, phones, encounters, etc.
 - Scoring combines **deterministic matches** (exact identifiers) and **probabilistic matches** (similar names, addresses).
-

✓ Scoring Components

1. Deterministic Signals (High Weight)

- Exact match on **MRN**, **SSN**, or enterprise ID → weight 0.8–1.0.
- Exact DOB match → weight 0.3–0.4.
- Exact phone/email match → weight 0.25–0.3.
- Same source system ID → weight 1.0 (if unique).

2. Probabilistic Signals

- **Name similarity** (Jaro-Winkler or Levenshtein):
 - Full name match → 0.8.
 - Similar first + last → 0.6.
- **Address similarity**:
 - Normalize (remove punctuation, abbreviations).
 - Compute token-based similarity → 0.4–0.6.
- **Gender match** → small boost (0.05–0.1).

3. Graph Connectivity

- Shared identifiers across multiple nodes → boost score.
- Shared phone/email nodes → boost.
- Shared household address → boost.
- Use **path-based scoring**: shorter path between two patients = higher confidence.

4. Optional Embedding Similarity

- Generate embeddings for combined patient profile text.
- Compute cosine similarity → add as a probabilistic component (0.2–0.3 weight).

✓ Example Weighted Formula

Plain Text ▾

```
1 Score = min(1.0,  
2   (0.4 * deterministic_hits) +  
3   (0.35 * name_similarity) +  
4   (0.15 * address_similarity) +  
5   (0.1 * embedding_similarity)  
6 )
```

- Deterministic hits can sum up weights for each exact match.
- Probabilistic components normalized to 0–1.

✓ Graph Query Pattern

In Neo4j (Cypher):

Cypher ▾

```
1 MATCH (p1:Patient)-[:HAS_IDENTIFIER]->(id)<-[:HAS_IDENTIFIER]-(p2:Patient)  
2 WHERE p1 <> p2  
3 WITH p1, p2, collect(id.value) AS sharedIds  
4 MATCH (p1)-[:HAS_ADDRESS]->(a1), (p2)-[:HAS_ADDRESS]->(a2)  
5 WITH p1, p2, sharedIds, a1, a2,  
6   apoc.text.jarowinklerDistance(p1.firstName + ' ' + p1.lastName,
```

```

7             p2.firstName + ' ' + p2.lastName) AS nameSim,
8     apoc.text.jarowinklerDistance(a1.fullAddress, a2.fullAddress) AS addrSim
9 RETURN p1.id, p2.id, sharedIds,
10        nameSim, addrSim,
11        CASE WHEN size(sharedIds) > 0 THEN 0.8 ELSE 0 END +
12        nameSim * 0.35 +
13        addrSim * 0.15 AS score
14 ORDER BY score DESC

```

✅ Best Practices

- Normalize all text (lowercase, remove punctuation, expand abbreviations).
 - Use phonetic encoding (Soundex, Metaphone) for names.
 - Store precomputed similarity scores as edge properties for faster queries.
 - Apply **thresholds**:
 - `score >= 0.85` → auto-merge.
 - `0.65 <= score < 0.85` → human review.
 - Maintain **audit trail** for merges and unmerges.
-

Do you want me to **provide a full Cypher script for building the graph schema and implementing this scoring logic**, or a **Python example using Neo4j driver to compute linkage scores and update Golden IDs**?

You asked about **alternatives to Azure AI Search** for patient matching and whether a **graph database** approach could work, given that your patient data will reside in **Azure Health Data Services (FHIR)** and flow into **Fabric bronze/silver lakehouses**.

✅ Alternatives to AI Search for EMPI

If you don't want to use Azure Cognitive Search for indexing and retrieval, here are common alternatives:

1. Graph Databases for Identity Resolution

- **Neo4j**, **TigerGraph**, **JanusGraph**, or **Azure Cosmos DB (Gremlin API)** are widely used for connected data scenarios.
- Why graph? Patient identity resolution is inherently a **relationship problem**: linking identifiers across systems, addresses, encounters, and even family relationships.
- Graph queries (e.g., `MATCH (p:Patient)-[:HAS_IDENTIFIER]->(id)` in Neo4j) allow you to traverse relationships efficiently and compute similarity scores.
- Use cases:
 - Detect clusters of records likely belonging to the same person.
 - Visualize linkage confidence and provenance.
- Considerations:
 - Graph DB excels when you need **complex joins** and **relationship analytics**.
 - For large-scale fuzzy matching, you'll still need **similarity scoring** (e.g., embeddings or phonetic algorithms) stored as node properties.

2. OpenEMPI (Open-source EMPI)

- Provides deterministic and probabilistic matching algorithms.

- Extensible architecture for plugging in custom matchers.
- Can integrate with FHIR-based workflows.
- Good if you want a **purpose-built EMPI engine** without building everything from scratch.

3. **ElasticSearch / Meilisearch**

- If you still want text-based fuzzy matching but without Azure AI Search, these open-source engines are popular.
- Combine with custom scoring logic for patient attributes.
- Works well for hybrid approaches (text + structured filters).

 Graph Database + Fabric Lakehouse Integration

Since you’re already using **Fabric medallion architecture** (bronze → silver → gold) and storing normalized FHIR data:

- **Bronze Layer:** Raw FHIR NDJSON from Azure Health Data Services.
- **Silver Layer:** Flattened relational FHIR tables (Patient, Encounter, Observation).
- **Gold Layer:** OMOP or custom analytics schema.

How to integrate a graph DB:

- Extract patient and identifier relationships from the **silver lakehouse**.
- Create nodes for:
 - Patient
 - Identifiers (MRN, SSN, EHR IDs)
 - Contact points (phone, email)
 - Addresses
- Create edges for:
 - `(:Patient)-[:HAS_IDENTIFIER]->(:Identifier)`
 - `(:Patient)-[:HAS_ADDRESS]->(:Address)`
- Store similarity scores as edge properties or node attributes.
- Use Fabric notebooks (Spark or T-SQL) to export data to **Neo4j** or **Cosmos DB Gremlin API**.
- Optionally, embed patient profiles using Azure OpenAI and store vectors in graph properties for advanced similarity queries.

 Pros & Cons of Graph vs Search

	≡ Feature	≡ Graph DB	≡ AI Search (Vector + Text)
1	Relationship modeling	Excellent	Limited
2	Fuzzy matching	Requires custom logic	Built-in with embeddings
3	Query complexity	Handles multi-hop queries easily	Hard for deep joins
4	Integration with Fabric	Needs ETL to graph DB	Native integration with OneLake

✅ Suggested Architecture Without AI Search

- Fabric Silver Lakehouse → ETL → Graph DB (Neo4j or Cosmos DB).
- Use Spark jobs in Fabric to:
 - Flatten FHIR resources.
 - Compute deterministic keys (DOB, MRN).
 - Generate similarity scores (Levenshtein, Jaro-Winkler).
- Store relationships in graph DB.
- Build a **matching service** that:
 - Runs Cypher/Gremlin queries for candidate retrieval.
 - Applies scoring logic for EMPI decisions.
- Optional: Use LLM + RAG over graph DB for advanced queries (e.g., "Find all patients linked by same phone and similar address").

Do you want me to draft a reference architecture diagram for a graph-based EMPI integrated with Fabric lakehouses, or provide sample code for exporting FHIR data from Fabric silver layer into Neo4j and building patient linkage queries?